

《LeetCode零基础指南》(第四讲) 指针

文章目录

零、了解网站

- 1、输入输出
- 2、刷题步骤
- 3、尝试编码
- 4、调试提交

一、概念定义

- 1、指针即地址
- 2、指针的定义
- 3、定义指针变量
- 4、取地址
- 5、数组的地址
- 6、解引用
- 7、内存申请
- 8、返回数组
- 9、范式
- 10、概念总结

二、题目分析

- 1、重新排列数组
- 2、数组串联
- 3、基于排列构建数组
- 4、前缀和
- 5、左旋字符串
- 6、IP地址无效化
- 7、替换空格
- 8、有多少小于当前数字的数字
- 9、打印1到最大的n位数
- 10、按既定顺序创建目标数组

三、推荐学习

四、课后习题

零、了解网站

1、输入输出

对于算法而言，就是给定一些输入，得到一些输出，在一般的在线评测系统中，我们需要自己手写输入输出函数（例如 C语言中的 `scanf` 和 `printf`），而在 **LeetCode** 这个平台，只需要实现它提供的函数即可。函数的传入参数就代表了算法的输入，而函数的返回值则代表了算法的输出。

2、刷题步骤

找到一道题，以 **两整数之和** 为例，如何把这道题通过呢？如下图所示：

力扣 学习 题库 讨论 竞赛 求职 商店

新功能 Plus 会员 我是面试官

题目描述 评论 (548) 题解 (510) 提交记录

371. 两整数之和

难度 中等 554

给你两个整数 a 和 b ，不使用运算符 $+$ 和 $-$ ，计算并返回两整数之和。

示例 1:

输入: $a = 1, b = 2$
输出: 3

示例 2:

输入: $a = 2, b = 3$
输出: 5

提示:

- $-1000 \leq a, b \leq 1000$

1 2 3 4 5 6

```
1 int getSum(int a, int b){
2
3
4 }
```

执行代码 提交

- 第 1 步: 阅读题目;
- 第 2 步: 参考示例;
- 第 3 步: 思考数据范围;
- 第 4 步: 根据题意, 实现函数的功能;
- 第 5 步: 本地数据测试;
- 第 6 步: 提交;

3、尝试编码

这个题目比较简单, 就是求两个数的和, 我们可以在编码区域尝试敲下这么一段代码。

```
1 int getSum(int a, int b){ // (1)
2     return a + b;        // (2)
3 }
```

- (1) 这里 `int` 是 C/C++ 中的一种类型, 代表整数, 即 Integer, 传入参数是两个整数;
- (2) 题目要求返回两个整数的和, 我们用加法运算符实现两个整数的加法;

4、调试提交

力扣

学习

题库

讨论

竞赛

求职

商店

新功能

Plus 会员

我是面试官

中

题目描述

评论 (548)

题解 (510)

提交记录

执行结果: 通过

显示详情

添加

执行用时: 0 ms, 在所有 C 提交中击败了 100.00% 的用户

内存消耗: 5.5 MB, 在所有 C 提交中击败了 17.94% 的用户

通过测试用例: 25 / 25

炫耀一下:

写题解, 分享我的解题思路

提交结果

执行用时

内存消耗

提交时间

备注

通过

通过

0 ms

5.5 MB

C

2021/11/22 22:33

添加

0 ms

5.2 MB

C

2021/11/22 13:21

添加

题目...

随...

上一题

371/2436

下一题

C

智能模式

模拟面试

i

1

2

3

4

int getSum(int a, int b){

return a + b;

}

测试用例

代码执行结果

调试器

Beta

已完成

执行用时: 0 ms

2

输入

45

289

3

输出

334

2

5

差别

预期结果

334

4

执行代码

提交

- 第 1 步: 实现加法, 将值返回;
- 第 2 步: 执行代码, 进行测试数据的调试;
- 第 3 步: 代码的实际输出;
- 第 4 步: 期望的输出, 如果两者符合, 则代表这一组数据通过 (但是不代表所有数据都能通过哦);
- 第 5 步: 尝试提交代码;
- 第 6 步: 提交通过, 撒花

这样, 我们就大致知道了一个刷题的步骤了。上一章我们了解了数组, 今天我们就来学习一下指针, 看看指针和数组到底有哪些千丝万缕的关系。

一、概念定义

1、指针即地址

计算机中所有的数据都必须放置在内存中, 不同类型的数据占用的字节数也不一样, 例如 32 位整型 `int` 占据 4 个字节, 64 位整型 `long long` 占据 8 个字节, 字符型 `char` 占据 1 个字节。

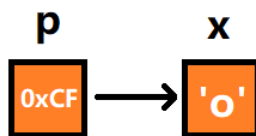
为了能够正确地访问这些数据, 必须为每个字节都编上编号, 每个字节的编号是唯一的。

我们把内存中字节的编号称为 **地址** 或 **指针**。地址从 0 开始依次递增, 对于 32 位环境下, 程序能够使用的内存为 4GB, 最小的地址为 0, 最大的地址为 0xFFFFFFFF。

2、指针的定义

在 C 语言中, 可以用一个变量来存放指针, 这种变量称为指针变量。指针变量的值, 通俗来说就是某一个变量的地址。

现在假设有一个 `char` 类型的变量 `x`, 它存储了字符 `'o'`, 并占用了地址为 `0xCF` 的内存 (地址通常用十六进制表示)。另外有一个指针变量 `p`, 它的值为 `0xCF`, 正好等于变量 `x` 的地址, 这种情况我们就称 `p` 指向了 `x`, 或者说 `p` 是指向变量 `x` 的指针。



其中 `p` 是指针变量名, `x` 是被指向的变量的变量名; `0xCF` 是指针变量 `p` 的值, 也是 `x` 的地址; `'o'` 是 `x` 的值。

3、定义指针变量

定义指针变量和普通变量类似, 只不过在变量名前面加上一个星号 `*` 即可。C 语言实现如下:

```
1 | DataType *dataName;
```

我们用到最多的指针变量类型为整型, 可以这么写:

```
1 | int *p;
```

对于字符类型的指针变量，可以这么写：

```
1 | char *p;
```

4、取地址

我们回到上面的问题，假设 x 是一个字符型变量， p 是 x 的地址，那么伪代码应该是这样的：

```
1 | char x = 'o';
2 | char *p = (x)的地址;
```

而实际写代码的时候，我们通过 `&` 来表示取地址符号，也就是代码可以写成：

```
1 | char x = 'o';
2 | char *p = &x;
```

5、数组的地址

对于数组而言，其中的元素的地址都是连续的，数组第一个元素的地址可以用数组名来表示，实现如下：

```
1 | int a[] = {5,2,0,1,3,1,4};
2 | int *p = a;
```

简而言之，`p` 指向数组第一个元素的地址，`a` 也是数组第一个元素的地址。

6、解引用

解引用是取地址的逆操作，即根据给定的地址，获取它的值。用 `*` 运算符来实现，如下：

```
1 | int a;
2 | int *p = &a;
3 | a == *p;
```

一句话解释，`p` 代表 `a` 的地址，`*p` 代表 `a` 的值；

7、内存申请

C语言中，利用函数 `malloc` 来申请内存，传入参数为申请内存的字节数，返回值为申请到的内存的首地址，是什么类型的地址，就要强转成什么类型，如下：

```
1 | int *p = (int *)malloc(1024);
```

`p` 代表的是申请到的内存，并且有效内存字节数为 1024。

如果我们要申请一个长度为 n 的整型数组的内存，可以这么写：

```
1 | int *p = (int *)malloc( sizeof(int) * n );
```

其中 `sizeof(int)` 表示的是一个 `int` 占用的字节数，那么一个长度为 n 的 `int` 类型的数组，需要的字节数自然就是 `sizeof(int) * n`。

8、返回数组

一般在刷题的时候，要求返回一个数组，这就比较尴尬，尴尬在哪里呢？

因为函数只能有一个返回值，而数组除了需要知道数组的首地址，还需要知道数组元素的个数。

所以，LeetCode 的做法是，通过函数的参数传递一个指针变量进来，然后让调用者去填充这个指针变量，如下：

```
1 | int *getList(int *nums, int numsSize, int *returnSize) {
2 |     // ...
3 | }
```

这里的 `returnSize` 是一个指针，它指向的就是一个代表数组长度的变量，它是没有赋值的，需要通过解引用对它进行赋值。

函数的调用方，其实是这么调用的：

```
1 | int a[7] = {5, 2, 0, 1, 3, 1, 4};
2 | int rSize;
3 | int *ret = getList(a, 7, &rSize);
```

这里的 `&rSize` 含义就是取了 `rSize` 的地址，传递给 `returnSize` 时，自然就成了指针。这样，在函数内部对 `*returnSize` 进行修改，自然就改了 `rSize` 的值，调用函数的一方自然就知道这个数组的长度了。

好了，今天的内容比较难，如果不懂，一定要在评论区留言告诉我，我会不断改善这块内容，争取能让更多的人听懂。

9、范式

所谓范式，就是规范写法的意思。就是按照这种规范写法写，能解决大部分问题。有关返回一维数组的问题，我们有通用范式，如下：

```
1  /**
2   * Note: The returned array must be malloced, assume caller calls free(). // (1)
3   */
4   int *func(int *nums, int numsSize, int *returnSize) { // (2)
5       int *ret = (int *)malloc( sizeof(int) * xxx ); // (3)
6       // TODO // (4)
7       *returnSize = xxx; // (5)
8       return ret; // (6)
9   }
```

- (1) 这一句话一般是系统提示你，需要申请内存，并且返回申请内存的首地址；
- (2) 这里会有两个返回值，一个是数组首地址（体现在返回值上），一个是数组大小（体现在指针传参上）；
- (3) 利用 `malloc` 申请一块自定义大小的内存；
- (4) 做你要做的事情；
- (5) 利用解引用，将需要返回的数组的长度告诉调用者；
- (6) 返回申请的数组内存的数组首地址；

10、概念总结

今天一下子接触到了太多概念，最后做个统一的总结：

名词	简介
指针	变量的地址
取地址	变量 -> 地址
解引用	地址 -> 变量
数组首地址	用数组名表示
范式	规范写法

二、题目分析

1、重新排列数组

给你一个数组 `nums`，数组中有 $2n$ 个元素，按 $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$ 的格式排列。请你将数组按 $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$ 格式重新排列，返回重新排列后的数组。

```
1  /**
2   * Note: The returned array must be malloced, assume caller calls free(). // (1)
3   */
4   int* shuffle(int* nums, int numsSize, int n, int* returnSize){ // (2)
5       int i;
6       int *ret = (int *)malloc( sizeof(int) * numsSize ); // (3)
7       for(i = 0; i < numsSize; ++i) { // (4)
8           if(i & 1) {
9               ret[i] = nums[n + i/2];
10            }else {
11                ret[i] = nums[(i+1)/2];
12            }
13        }
14        *returnSize = numsSize; // (5)
15        return ret; // (6)
16    }
```



- (1) 这句话是核心，返回的数组必须是通过 `malloc` 进行内存分配的，因为调用者会对它进行 `free` 的清理操作；
- (2) 这个函数的返回值是 `int *`，即返回的是一个数组的首地址，但是数组的大小也需要让调用者知道，所以通过一个传参 `returnSize` 来传递出去；
- (3) 申请一个长度为 `numsSize` 的数组，首地址为 `ret`；
- (4) 根据题目要求，将 `nums` 的数据填充到 `ret` 中；
- (5) 最后，别忘了利用解引用，将需要返回的数组的长度告诉调用者；
- (6) 返回你申请的数组的首地址；

2、数组串联

给你一个长度为 n 的整数数组 `nums`。请你构建一个长度为 $2n$ 的答案数组 `ans`，数组下标从 0 开始计数，对于所有 $0 \leq i < n$ 的 i ，满足下述所有要求：

$$\begin{aligned}ans[i] &== nums[i] \\ans[i + n] &== nums[i]\end{aligned}$$

具体而言，`ans` 由两个 `nums` 数组 串联 形成。返回数组 `ans`。

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4 int* getConcatenation(int* nums, int numsSize, int* returnSize){
5     int i;
6     int *ret = (int *)malloc(2*numsSize*sizeof(int)); // (1)
7     for(i = 0; i < numsSize; ++i) {
8         ret[i+numsSize] = ret[i] = nums[i];          // (2)
9     }
10    *returnSize = 2 * numsSize;                       // (3)
11    return ret;
12 }
```

- (1) 这题和上题的不同之处在于：申请一个长度为 $2 * \text{numsSize}$ 的数组，首地址为 `ret`；
- (2) 根据题目要求，将 `nums` 的数据填充到 `ret` 中；
- (3) 别忘了利用解引用，将需要返回的数组的长度告诉调用者；
- (4) 返回你申请的数组的首地址；

3、基于排列构建数组

给你一个从 0 开始的排列 `nums`（下标也从 0 开始）。请你构建一个同样长度的数组 `ans`，其中，对于每个 $i(0 \leq i < \text{nums.length})$ ，都满足 `ans[i] = nums[nums[i]]`。返回构建好的数组 `ans`。

从 0 开始的排列 `nums` 是一个由 0 到 $\text{nums.length} - 1$ （0 和 $\text{nums.length} - 1$ 也包含在内）的不同整数组成的数组。

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4 int* buildArray(int* nums, int numsSize, int* returnSize){
5     int i;
6     int *ret = (int *)malloc( sizeof(int) * numsSize );
7     for(i = 0; i < numsSize; ++i) {
8         ret[i] = nums[ nums[i] ];          // (1)
9     }
10    *returnSize = numsSize;
11    return ret;
12 }
```

- (1) 除了这一步不同，其它都是上文提到的范式的内容。

4、前缀和

给你一个数组 `nums`。数组「动态和」的计算公式为：

$$\text{runningSum}[i] = \text{sum}(\text{nums}[0] \dots \text{nums}[i])$$

请返回 `nums` 的动态和。

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4 int* runningSum(int* nums, int numsSize, int* returnSize){
5     int i;
6     int *ret = (int *)malloc( numsSize * sizeof(int) );
7     for(i = 0; i < numsSize; ++i) {      // (1)
8         ret[i] = nums[i];
9         if(i) {
10            ret[i] += ret[i-1];
11        }
12    }
13    *returnSize = numsSize;
14    return ret;
15 }
```

- (1) 当 $i = 0$ 时, $ret[0] = nums[0]$; 当 $i > 0$ 时, $ret[i] = ret[i - 1] + nums[i]$, $O(1)$ 的时间复杂度计算每个元素的动态和。

5、左旋字符串

字符串的左旋转操作是把字符串前面的若干个字符转移到字符串的尾部。请定义一个函数实现字符串左旋转操作的功能。比如，输入字符串 "abcdefg" 和数字 2，该函数将返回左旋转两位得到的结果 "cdefgab"。

```
1 char* reverseLeftWords(char* s, int k){
2     int i;
3     int n = strlen(s);
4     char *ret = (char *)malloc( (n + 1) * sizeof(char) );    // (1)
5     for(i = 0; i < n; ++i) {
6         ret[i] = s[(i + k) % n];                            // (2)
7     }
8     ret[n] = '\0';                                          // (3)
9     return ret;
10 }
```

- (1) 注意，字符串需要多申请一个自己的内存，用来存放结尾标记 `\0`；
- (2) 把字符串首位相接，循环取模；
- (3) 加上结尾标记以后，调用者就不需要知道实际字符串有多长，可以通过 `strlen` 来获取了；

6、IP地址无效化

给你一个有效的 IPv4 地址 address，返回这个 IP 地址的无效化版本。所谓无效化 IP 地址，其实就是用 "[.]" 代替了每个 "."。

```
1 char * defangIPaddr(char * address){
2     char *ret = (char *)malloc(1000 * sizeof(char));    // (1)
3     int returnSize = 0;
4     int i;
5     for(i = 0; address[i]; ++i) {                        // (2)
6         if(address[i] == '.') {
7             ret[ returnSize++ ] = '[';
8             ret[ returnSize++ ] = '.';
9             ret[ returnSize++ ] = ']';
10        }else {
11            ret[ returnSize++ ] = address[i];
12        }
13    }
14    ret[ returnSize ] = '\0';                            // (3)
15    return ret;
16 }
```



- (1) 直接在原字符串代替不好做，所以我们可以申请一个新的字符串，长度可以定长一点关系不大；
- (2) 然后遇到 '.' 字符，我们就在返回的字符串上加上三个字符： '['、 '.'、 ']'，否则添加原字符；
- (3) 最后，在字符串末尾添加一个 '\0' 表示字符串结尾；

7、替换空格

请实现一个函数，把一个长度不大于 10000 的字符串 s 中的每个空格替换成 "%20"。

```
1 char* replaceSpace(char* s){
2     char *ret = malloc( sizeof(char) * 30001 ); // (1)
3     int i, retSize = 0;
4     for(i = 0; s[i]; ++i) {
5         if(s[i] == ' ') {
6             ret[retSize++] = '%';
7             ret[retSize++] = '2';
8             ret[retSize++] = '0';
9         }else {
10            ret[retSize++] = s[i];
11        }
12    }
13    ret[retSize] = '\0';
14    return ret;
15 }
```

- (1) 和上面那题的思路一模一样，只是需要注意字符串长度可能很长，如果 10000 个空格的话，替换完最多 30001 个字符；

8、有多少小于当前数字的数字

给你一个数组 `nums`，对于其中每个元素 `nums[i]`，请你统计数组中比它小的所有数字的数目。换言之，对于每个 `nums[i]` 你必须计算出有效的 j 的数量，其中 j 满足 $j \neq i$ 且 $nums[j] < nums[i]$ 。以数组形式返回答案。

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4 int* smallerNumbersThanCurrent(int* nums, int numsSize, int* returnSize){
5     int *ret = (int *)malloc(numsSize * sizeof(int));
6     int i, j;
7     for(i = 0; i < numsSize; ++i) {
8         ret[i] = 0; // (1)
9         for(j = 0; j < numsSize; ++j) {
10             if(nums[j] < nums[i]) { // (2)
11                 ++ret[i];
12             }
13         }
14     }
15     *returnSize = numsSize;
16     return ret;
17 }
```



- (1) `ret[i]` 表示的是满足所有小于 $nums[j] < nums[i]$ 的数的数量；
- (2) 对于每个 `ret[i]`，循环遍历 `nums` 数组，满足条件则加一；

9、打印1到最大的n位数

输入数字 n ，按顺序打印出从 1 到最大的 n 位十进制数。比如输入 3，则打印出 1、2、3 一直到最大的 3 位数 999。

```
1 int* printNumbers(int n, int* returnSize){
2     int i;
3     int f = 1;
4     int *ret;
5
6     for(i = 0; i < n; ++i) {
7         f *= 10;
8     }
9     --f;
10    *returnSize = f; // (1)
11    ret = (int *)malloc( f * sizeof(int) );
12    for(i = 1; i <= f; ++i) {
13        ret[i-1] = i; // (2)
14    }
15    return ret;
16 }
```



- (1) n 位数总共有 $f = 10^n - 1$ 个；
- (2) 第0个元素放1，第1个元素放2，所以这里需要做个-1的偏移；

10、按既定顺序创建目标数组

给你两个整数数组 `nums` 和 `index`。你需要按照以下规则创建目标数组：

- 1) 目标数组 `target` 最初为空。
 - 2) 按从左到右的顺序依次读取 `nums[i]` 和 `index[i]`，在 `target` 数组中的下标 `index[i]` 处插入值 `nums[i]`。
 - 3) 重复上一步，直到在 `nums` 和 `index` 中都没有要读取的元素。
- 请你返回目标数组。题目保证数字插入位置总是存在。

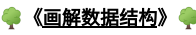
```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
```

```
3  */
4  int* createTargetArray(int* nums, int numsSize, int* index, int indexSize, int* returnSize){
5      int len = 0;
6      int i, j, ins, idx;
7      int *ret = (int *) malloc( sizeof(int) * numsSize );
8      for(i = 0; i < numsSize; ++i) {
9          idx = index[i];
10         ins = nums[i];
11         for(j = len; j > idx; --j) { // (1)
12             ret[j] = ret[j-1];
13         }
14         ret[idx] = ins;           // (2)
15         ++len;                   // (3)
16     }
17     *returnSize = len;
18     return ret;
19 }
```



- (1) 由于需要对元素进行插入，所以，后面的元素需要往后挪一个位置，数组的插入可以参见 顺序表的插入。
- (2) 将元素放入待插入位置；
- (3) 插入完毕，数组长度势必加一；

三、推荐学习



(1-1) 顺序表

四、课后习题

课后所有习题，在上文都能找到答案，所以务必将 [必做] 为 1 的题全部刷完。

第五天了，感觉如何？如果指针已经理解，那么恭喜你，C语言的最难点就告一段落了，后面就是搭积木了。其实举办这个九日集训的目的，最终还是为了让大家养成习惯，习惯刷题并且爱上它，我相信，到了后面你自己也会忍不住自己找题来刷。

坚持！加油！你可以的！

序号	题目链接	难度	必做
1	重新排列数组	★☆☆☆☆	1
2	数组串联	★☆☆☆☆	1
3	基于排列构建数组	★☆☆☆☆	1
4	一维数组的动态和	★☆☆☆☆	1
5	左旋字符串	★☆☆☆☆	1
6	IP 地址无效化	★☆☆☆☆	0
7	替换空格	★☆☆☆☆	0
8	有多少小于当前数字的数字	★☆☆☆☆	0
9	打印从1到最大的n位数	★☆☆☆☆	0
10	按既定顺序创建目标数组	★★☆☆☆	0

